IgnitionAI

Technical Architecture Blueprint

AI Diagnostic Assistant — ICE Passenger Vehicles, India

Version 1.0 | June 2026 | Confidential
Prepared by: Shashank A, Founder & CEO
Target Build Window: 8 Weeks (Prototype → Field-Ready MVP)

Table of Contents

1	System Philosophy & Design Principles	3
2	High-Level Architecture Overview	3
3	Layer 1 — OBD-II Data Ingestion	4
4	Layer 2 — Data Pipeline & Normalisation	5
5	Layer 3 — Diagnostic AI Engine	6
6	Layer 4 — API Gateway & Backend Services	9
7	Layer 5 — Frontend (Mobile + Web)	10
8	Layer 6 — Feedback & Active Learning Loop	11
9	Data Schema Design	12
10	Infrastructure & DevOps	13
11	Security & Privacy Architecture	14
12	Technology Stack Master Table	15
13	8-Week Build Roadmap	16
14	Team Responsibilities & Handoff Protocol	17
15	Known Risks & Engineering Mitigations	18

© 2026 IgnitionAI — Confidential & Proprietary

1. System Philosophy & Design Principles

IgnitionAI is built on a single architectural north star: **every software decision must reduce time-to-correct-diagnosis for a mechanic with a mid-range Android phone and a ■500 ELM327 dongle.** Architectural elegance that does not serve this goal is waste. The following principles govern every layer:

- **Offline-first, cloud-enhanced**: The MVP must work without internet. Cloud improves latency and accuracy but must never be a hard dependency.
- **Rule-based core, ML-augmented**: The diagnostic engine ships with deterministic automotive rules. ML models are layered on top as confidence improves with data. No ML required for v1 correctness.
- **Failure-transparent**: When the system is uncertain, it says so with a confidence score. A wrong confident answer is worse than an honest 'I don't know'. This is also legal risk management.
- **Modular monolith first**: Start as a single deployable backend (FastAPI). Split into microservices only when a specific service needs independent scaling. Premature microservices will kill your 8-week timeline.
- **Data collection as a first-class feature**: Every diagnosis session must record inputs, outputs, and technician feedback. The feedback loop is the product moat. Instrument from day 1.
- **No vendor lock-in on inference**: Abstract LLM calls behind an inference interface. You can swap OpenAI → Gemini → local Mistral without rewriting the diagnostic engine.

2. High-Level Architecture Overview

The system has six discrete layers. Data flows from the physical vehicle upward through normalisation, reasoning, API, and presentation layers, then loops back through a feedback mechanism that continuously improves the diagnostic engine.

Layer	Name	Primary Technology	Output Artifact
L1	OBD-II Ingestion	Mobile App (Python-OBD / ELM327 BT)	Raw DTC + PID stream
L2	Data Pipeline	Python (FastAPI worker, Pydantic)	Normalised FaultSession JSON
L3	Diagnostic AI Engine	Python (Rule Engine + optional LLM)	Ranked RootCause list
L4	API Gateway	FastAPI + JWT auth	REST JSON responses
L5	Frontend	Flutter (mobile) + React (web)	UX for mechanics & owners
L6	Feedback Loop	PostgreSQL + background job	Outcome labels for model training

All layers communicate over HTTPS. The mobile app talks directly to the API gateway for live sessions, and to a local SQLite cache for offline operation. The backend is stateless per request; session state lives in PostgreSQL.

© 2026 IgnitionAI — Confidential & Proprietary

3. Layer 1 — OBD-II Data Ingestion

3.1 Hardware Interface

The ingestion layer targets ELM327-compatible Bluetooth (v2.0 Classic, not BLE) dongles, which retail for ■400–800 on Amazon India. The Flutter app opens a Bluetooth RFCOMM socket and communicates with the dongle using raw AT commands followed by OBD-II Mode requests.

Supported OBD-II Modes (MVP)

- Mode 01 — Live PIDs: RPM, MAF, MAP, TPS, ECT, IAT, O2 sensors, fuel trim (STFT/LTFT), battery voltage
- Mode 02 — Freeze Frame: PID snapshot at moment of DTC set
- Mode 03 — Stored DTCs: all active P/B/C/U codes
- Mode 04 — Clear DTCs (with explicit user confirmation — liability gate)
- Mode 06 — On-Board Monitoring: misfire counters, O2 monitor results
- Mode 09 — Vehicle info: VIN (if ECU supports), calibration IDs

3.2 AT Command Sequence (Startup Protocol)

Command	Purpose
`ATZ`	Reset ELM327 chip
`ATE0`	Echo off
`ATL0`	Linefeeds off
`ATH1`	Headers on (needed for multi-ECU)
`ATSP0`	Auto-detect OBD protocol (ISO 9141, KWP2000, CAN 11/29-bit)
`0100`	Probe supported PIDs, confirm OBD link
`0902`	Request VIN — graceful fail if unsupported
`03`	Read all stored DTCs
`0120, 0140, 0160`	Extended PID support bitmaps

3.3 Key PIDs for No-Start / Misfire Diagnosis

PID Hex	Parameter	Range	Diagnostic Relevance
0C	RPM	0–8000 rpm	Cranking? Running? Stall speed?
11	TPS %	0–100%	Stuck open/closed → flood/lean
10	MAF g/s	0–655 g/s	Low MAF → air leak, dirty sensor
0B	MAP kPa	0–255 kPa	High at idle → vacuum leak

© 2026 IgnitionAI — Confidential & Proprietary

0C	ECT °C	-40–215°C	Cold start richness check
06	STFT %	-100–+100%	>10% lean, <-10% rich
07	LTFT %	-100–+100%	Persistent fuel correction
12	Sec air	enum	Air injection status
2F	Fuel lvl	0–100%	Obvious — eliminates fuel starvation
42	Ctrl V	6–16 V	<11.5V → charging/battery issue
0E	Ign adv	-64–63.5°	Timing retard → knock/cam issue

© 2026 IgnitionAI — Confidential & Proprietary

4. Layer 2 — Data Pipeline & Normalisation

4.1 FaultSession Data Model

Every diagnostic interaction is serialised as a **FaultSession** object. This is the atomic unit of data across ingestion, reasoning, storage, and feedback. Defining this schema correctly is the single most important data decision in the project.

`session_id` `UUID` Primary key, generated at intake

`created_at` `datetime` UTC timestamp

`vehicle_id` `UUID` FK Linked to Vehicle table

`mechanic_id` `UUID` FK Linked to User table

`symptoms_raw` `text[]` Free-text symptom strings from intake form

`symptoms_coded` `text[]` Normalised symptom codes (SNOBD standard)

`dtcs` `DTC[]` List of DTC objects (code, type, freeze_frame)

`live_pids` `PIDSnapshot` Dict of PID hex → (value, unit, timestamp)

`environment` `EnvContext` Ambient temp, fuel type, city, odometer

`diagnosis_output` `DiagOutput` AI engine result (ranked causes + test plan)

`feedback` `Feedback?` Post-repair outcome (nullable until resolved)

`status` `enum` OPEN / IN_PROGRESS / RESOLVED / ABANDONED

4.2 DTC Object Schema

Field	Type	Description
code	str	e.g. 'P0300' — standardised 5-char
dtc_type	enum	STORED / PENDING / PERMANENT
system	enum	POWERTRAIN / BODY / CHASSIS / NETWORK
freeze_frame	dict	PID values at DTC set moment (Mode 02)
cleared	bool	Whether code was already cleared
occurrences	int	From Mode 06 counters if available

4.3 Normalisation Pipeline Steps

- 1. **Protocol decode**: raw ELM327 hex response → typed Python dict via python-OBD decoders
- 2. **Unit conversion**: all values stored in SI units (°C, kPa, g/s, rpm). Display conversion at UI layer only.
- 3. **Sanity check**: RPM > 0 while engine_running=False → flag as dongle error, do not pass to AI engine
- 4. **Vehicle context join**: merge vehicle_id → make/model/year/engine code from Vehicle table
- 5. **DTC classification**: P0xxx powertrain-generic, P1xxx OEM-specific (flag as lower confidence)
- 6. **Symptom NLP**: free text → symptom codes using keyword matcher (regex v1, fine-tuned BERT v2)
- 7. **Freeze frame alignment**: match freeze frame PID set to live PID set, compute delta

© 2026 IgnitionAI — Confidential & Proprietary

- 8. **Session hash**: SHA-256 of (vehicle_id + dtcs + live_pids timestamp) for dedup detection

© 2026 IgnitionAI — Confidential & Proprietary

5. Layer 3 — Diagnostic AI Engine

The AI engine is the core IP of IgnitionAI. It consumes a normalised FaultSession and produces a **DiagnosisOutput**: a ranked list of root-cause hypotheses, each with a confidence score, supporting evidence, and a minimal test plan. The engine is designed as a pipeline of three modules that can each be independently upgraded without breaking the others.

5.1 Module A — Rule-Based Causal Mapper (Ships in Week 4)

This is the heart of v1. It is a deterministic Python module: given a set of DTCs and PID readings, it applies automotive expert rules to generate hypotheses. No ML, no model training required. This is the module you build first.

Rule Structure: Each rule is a Python dataclass:

```
@dataclass
class DiagRule:
    rule_id: str # e.g. 'R-P0300-01'
    trigger_dtcs: list[str] # ['P0300', 'P0301']
    pid_conditions: list[Condition] # PID checks
    symptom_match: list[str] # ['rough_idle', 'misfire']
    hypothesis: str # 'Ignition coil failure – Cyl 1'
    base_confidence: float # 0.0–1.0
    test_plan: list[TestStep]
    vehicle_filter: VehicleFilter # restrict by make/engine
    evidence_keys: list[str] # which PIDs support this rule
```

Priority DTC Clusters to implement first (covers ~65% of no-start cases in India):

Cluster	DTCs	Primary Hypotheses	Key PIDs
Misfire	P0300–P0308	Coil, plug, injector, compression, timing	RPM fluctuation, Mode 06 misfire counters
Fuel System Rich	P0172, P0175	Stuck injector, EVAP purge, O2 sensor	LTFT < -10%, fuel pressure if available
Fuel System Lean	P0171, P0174	Vacuum leak, dirty MAF, weak fuel pump	MAF low, MAP high, LTFT > +10%
Crank/Cam Sensor	P0335, P0340	Sensor failure, reluctor ring, wiring	RPM=0 while cranking, erratic signal
Throttle/Pedal	P0120–P0124	TPS sensor, wiring, throttle body carbon	TPS%, idle RPM instability
Cooling/Temp	P0115–P0119	ECT sensor, thermostat, coolant level	ECT reading, startup enrichment
Battery/Charging	P0560–P0563	Alternator, belt, battery, ground strap	PID 42 voltage < 11.5V
EVAP System	P0440–P0457	Purge valve, vent valve, fuel cap, hose	Idle LTFT spikes, fuel smell symptom
O2/AFR Sensor	P0130–P0167	Sensor aging, exhaust leak, wiring	O2 oscillation pattern analysis

© 2026 IgnitionAI — Confidential & Proprietary

5.2 Module B — Bayesian Confidence Updater (Integrates Week 6)

Once Module A generates initial hypotheses, Module B adjusts confidence scores using a Naive Bayes update. For each hypothesis H given evidence E:

P(H|E) = P(E|H) × P(H) / P(E)

- **P(H)** — prior probability of each hypothesis: seeded from automotive repair statistics (IAM, MEMA reports). Initially hardcoded, later learned from feedback loop.
- **P(E|H)** — likelihood of evidence given hypothesis: e.g. P(LTFT > 10% | vacuum_leak) = 0.82. Built from expert rules initially.
- **Evidence inputs**: LTFT/STFT values, freeze frame RPM, ambient temperature, fuel type (CNG/petrol/diesel), vehicle age, odometer bucket (0–50k / 50–100k / 100k+), city (proxy for fuel quality).
- **Multi-hypothesis update**: all hypotheses normalise to sum to 1.0. Display top-5 with confidence bars.
- **Cold-start fallback**: if P(H) is unknown for a DTC code, fall back to Module A rule confidence × 0.7 (discount for uncertainty).

5.3 Module C — LLM Reasoning Layer (Optional Enhancement, Week 7–8)

For complex cases where Module A/B return confidence < 0.45, or for generating natural-language explanations, an LLM is invoked via API. This is **not required for the MVP to function** — it enhances output quality for edge cases.

LLM Integration Design

• Model: GPT-4o-mini (cost-optimised) or claude-haiku-4 via Anthropic API. Abstract behind InferenceClient interface.

• Prompt construction: System prompt = automotive expert persona + vehicle context. User prompt = serialised FaultSession JSON + 'Generate ranked hypotheses with test steps.'

• Output parsing: Force JSON output via response_format={'type':'json_object'}. Schema-validate with Pydantic before accepting.

• Guardrails: Never output 'replace part X' without a test step first. Never claim >95% confidence. Always include a disclaimer string.

• Cost control: Only invoke for sessions where rule engine confidence < 0.45. Cache responses by session_hash (Redis, 24h TTL). Budget ~■2-3 per complex session.

• Fallback: If LLM API fails, return Module A/B result with a UI flag 'Advanced analysis unavailable - basic diagnosis shown'.

5.4 Test Plan Generator

Each hypothesis includes an ordered list of TestStep objects. Steps are ordered by **information gain / effort ratio** — cheapest, fastest, most-ruling tests first. This is deterministic logic, not ML.

Field	Type	Description
step_id	str	Unique ID for cross-reference
title	str	e.g. 'Check fuel pressure at rail'
tool_req	str[]	e.g. ['fuel pressure gauge', 'multimeter']
procedure	str	Step-by-step instruction text (Markdown)
pass_crit	str	e.g. 'Pressure 2.8–3.2 bar at idle'
fail_action	str	What to conclude if step fails

© 2026 IgnitionAI — Confidential & Proprietary

time_est	int	Estimated minutes
safety_note	str?	Mandatory safety warning if applicable
alt_if_no_tool	str?	Alternative test if primary tool unavailable

© 2026 IgnitionAI — Confidential & Proprietary

6. Layer 4 — API Gateway & Backend Services

6.1 Framework & Structure

Backend: **FastAPI 0.111+** on Python 3.11. Chosen for automatic OpenAPI docs, async support, and Pydantic v2 integration. Deployed on Railway.app (free tier → paid at pilot stage) via Docker.

Method	Endpoint	Description	Auth
POST	/auth/register	Workshop/user registration	Public
POST	/auth/token	JWT login (OAuth2 password flow)	Public
POST	/sessions/start	Begin diagnostic session, return session_id	Auth
POST	/sessions/{id}/obd	Push OBD data batch (DTCs + PIDs)	Auth
GET	/sessions/{id}/diagnosis	Trigger AI engine, return DiagOutput	Auth
POST	/sessions/{id}/feedback	Submit repair outcome feedback	Auth
GET	/vehicles/	List vehicles for account	Auth
POST	/vehicles/	Add vehicle (VIN or manual make/model/year)	Auth
GET	/dtcs/{code}	Lookup DTC definition + common causes	Auth
GET	/health	Liveness probe for deployment	Public

6.2 Authentication & Multi-Tenancy

- JWT tokens with 8-hour expiry. Refresh tokens with 30-day expiry stored in Redis.
- Tenant isolation by workshop_id. Every DB query includes workshop_id filter — no cross-tenant data leakage possible at query level.
- Role-based: OWNER (full access), MECHANIC (sessions + vehicles), VIEWER (read-only reports).
- Rate limiting: 100 req/min per IP (Slowapi middleware) to prevent abuse during free tier.
- Password hashing: bcrypt with cost factor 12.

6.3 Database Architecture

Primary DB: PostgreSQL 16 on Railway (or Supabase free tier). **Cache:** Redis 7 (Upstash free tier) for session state, LLM response cache, rate limiting. **Mobile offline:** SQLite via Drift (Flutter) — syncs to Postgres on reconnect.

Table	Key Columns
`workshops`	`id, name, city, plan_tier, created_at`

© 2026 IgnitionAI — Confidential & Proprietary

users	id, workshop_id, email, password_hash, role, created_at
vehicles	id, workshop_id, vin, make, model, year, engine_code, fuel_type, odometer
fault_sessions	id, vehicle_id, mechanic_id, status, created_at, resolved_at
session_dtcs	id, session_id, code, dtc_type, freeze_frame JSONB
session_pids	id, session_id, pid_hex, value, unit, captured_at
diagnosis_outputs	id, session_id, ranked_causes JSONB, test_plan JSONB, engine_version, created_at
feedback_records	id, session_id, confirmed_cause, parts_replaced JSONB, time_to_fix_min, tech_rating

© 2026 IgnitionAI — Confidential & Proprietary

7. Layer 5 — Frontend (Mobile + Web)

7.1 Mobile App (Flutter)

Target: Android 8.0+ (API 26+). Flutter chosen for single codebase maintainability with a 2-person team. The app handles BT OBD connection, offline data capture, and the primary mechanic workflow.

Screen	Purpose	Key Components
Splash/Login	JWT auth	Email + password, remember me
Dashboard	Session list + vehicle list	Pull-to-refresh, add vehicle FAB
New Session	Intake form	Symptom selector, vehicle picker, environment fields
OBD Connect	BT scanning + connection	Device list, connection status, live voltage readout
Live PIDs	Real-time PID monitor	Scrollable PID cards with colour-coded ranges
Diagnosis Result	Ranked causes + confidence	Card stack, expand for evidence, test plan steps
Test Steps	Guided procedure	Checklist with pass/fail capture, photo note option
Feedback	Post-repair outcome entry	Cause confirm, parts used, time taken, star rating
Reports	Session history + export	List view, PDF export via share sheet

7.2 Key Flutter Packages

Package	Purpose
`flutter_bluetooth_serial`	ELM327 Classic BT RFCOMM socket
`drift`	Local SQLite ORM for offline storage
`dio + retrofit`	Typed HTTP client for API calls
`riverpod`	State management (preferred over BLoC for 2-person team)
`go_router`	Declarative navigation
`fl_chart`	PID trend charts
`pdf + printing`	In-app report generation
`flutter_secure_storage`	JWT token secure storage

7.3 Web Console (React)

Secondary deliverable — targets workshop owners and fleet managers on desktop. React 18 + TypeScript + TanStack Query + shadcn/ui. Read-only analytics dashboard in v1: session history, diagnostic accuracy metrics, repair time trends. Authentication shares the same JWT backend.

© 2026 IgnitionAI — Confidential & Proprietary

8. Layer 6 — Feedback & Active Learning Loop

The feedback loop is what separates IgnitionAI from a static rule book. Every resolved session that receives a technician feedback record becomes a training datapoint for improving the Bayesian priors and, eventually, fine-tuning the LLM.

- 1. **Session resolved**: mechanic marks case RESOLVED and submits confirmed_cause + parts_replaced
- 2. **Outcome labelling**: background job compares AI's top-1 hypothesis against confirmed_cause → label: CORRECT / PARTIALLY_CORRECT / WRONG
- 3. **Prior update**: for each DTC cluster, update P(H) = (count_correct + alpha) / (total + alpha*k) using Laplace smoothing
- 4. **Evidence weight update**: if diagnosis was wrong, which PIDs were present but misleading? Downweight those P(E|H) estimates
- 5. **Weekly batch export**: anonymised FaultSession + label → CSV pushed to Google Drive for offline analysis
- 6. **Model retraining trigger**: when feedback_records > 500 for a DTC cluster, queue a Bayesian prior refresh job
- 7. **Diagnostic accuracy dashboard**: internal Metabase/Grafana showing accuracy by DTC cluster, vehicle make, city

Minimum Viable Feedback UX

- 3-tap confirmation: 'Was the AI's #1 suggestion correct?' → Yes / Partially / No
- If No: show top-5 hypotheses + 'Other' → mechanic selects actual cause
- Parts used: pre-populated from test plan parts list, tap to confirm or edit
- Time saved: 'How long did this diagnosis take vs your normal process?' → slider
- This UX takes < 90 seconds — critical for adoption by time-pressured mechanics

© 2026 IgnitionAI — Confidential & Proprietary

9. Data Schema Design

9.1 Vehicle Taxonomy

Vehicle data is the foundation of accurate diagnosis. The Vehicle table uses a hierarchical key: **make** → **model** → **year_range** → **engine_code**. Pre-seed with the top 20 models representing ~70% of Indian passenger car parc:

Make	Models	Engine Codes
Maruti Suzuki	Swift / Dzire / Baleno	K10C/K12B/K12N (1.0–1.2 DualJet)
Maruti Suzuki	Alto 800 / Alto K10	F8D / K10B
Maruti Suzuki	Wagon R	K10C Mild Hybrid
Hyundai	i10 / i20 / Grand i10	Kappa 1.0/1.2, Nu 1.2
Hyundai	Venue / Creta	Kappa 1.0T, Nu 1.5
Tata Motors	Nexon / Altroz / Punch	Revotron 1.2T, 1.5
Honda	City / Amaze	L15Z4, L15B6 (5th/6th gen)
Toyota / Suzuki	Innova Crysta	2GD-FTV diesel
Kia	Seltos / Sonet	Smartstream G1.5, G1.0T
Renault	Kwid / Triber	BR10 0.8/1.0

9.2 DTC Knowledge Base Schema

Static table, seeded once. The MVP should cover P0xxx and P1xxx for the target vehicle set. Source: SAE J2012 standard (public), supplemented by OEM-specific codes.

Column	Type	Notes
dtc_code	TEXT PK	e.g. 'P0300'
short_desc	TEXT	SAE standard description
long_desc	TEXT	Detailed explanation of fault
affected_systems	TEXT[]	['ignition','fuel','sensor']
likely_causes	TEXT[]	Ordered by frequency in field data
quick_checks	TEXT[]	First 3 checks any mechanic should do
vehicle_scope	TEXT	ALL / PETROL_ONLY / DIESEL_ONLY / specific makes
severity	ENUM	LOW / MEDIUM / HIGH / SAFETY_CRITICAL

© 2026 IgnitionAI — Confidential & Proprietary

10. Infrastructure & DevOps

10.1 Deployment Architecture (Months 1–2)

Keep infrastructure dead simple for the prototype. Single server, managed database, no Kubernetes. Scale later — don't prematurely complicate the stack.

Component	Service	Tier	Monthly Cost
Backend API	Railway.app	Starter ($5/mo)	~■420
PostgreSQL	Railway.app / Supabase	Free (500MB)	■0
Redis Cache	Upstash	Free (10k cmd/day)	■0
Static Assets	Cloudflare R2	Free (10GB)	■0
Domain + SSL	Cloudflare	Free tier + ■800/yr domain	~■70/mo
Error Monitoring	Sentry	Free tier (5k errors/mo)	■0
LLM API	OpenAI / Anthropic	Pay-per-use (~$0.001/session)	~■84/100 sessions
CI/CD	GitHub Actions	Free (2000 min/mo)	■0
APM	Grafana Cloud	Free tier	■0

10.2 Docker Setup

Single docker-compose.yml for local development:

```
services:
  api:
    build: .
    ports: ['8000:8000']
    env_file: .env
    depends_on: [db, redis]
  db:
    image: postgres:16-alpine
    volumes: [pg_data:/var/lib/postgresql/data]
  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 128mb
```

10.3 CI/CD Pipeline (GitHub Actions)

- On push to main: run pytest (unit + integration), ruff linting, mypy type-check
- On PR: run full test suite + coverage report (target: >70% for diagnostic engine)
- On tag vX.Y.Z: build Docker image → push to GitHub Container Registry → Railway auto-deploy
- Secrets: DATABASE_URL, REDIS_URL, JWT_SECRET, OPENAI_API_KEY stored in GitHub Secrets
- Branch strategy: main (production), dev (integration), feature/* (per-intern branches)

© 2026 IgnitionAI — Confidential & Proprietary

11. Security & Privacy Architecture

Automotive diagnostic data is sensitive (vehicle location proxies, fault history, driving behaviour). Even at MVP stage, implement these controls — retrofitting security is significantly harder than building it in from the start.

Security Controls	Privacy Controls
Transport: TLS 1.3 minimum. HSTS headers. Certificate pinning in mobile app for API calls.	**Data minimisation**: Only collect PIDs needed for diagnosis. No GPS, no microphone, no contact list access.
Data at rest: PostgreSQL column-level encryption for PII fields (email, phone) using pgcrypto.	**Consent**: Workshop signs data processing agreement at onboarding. Vehicle owner consent captured for fleet deployments.
API keys: All secrets in environment variables. Zero secrets in Git. .env.example in repo with placeholder values only.	**Anonymisation**: Before any data leaves individual workshop context (for global learning), strip: VIN, exact odometer, workshop_id. Replace with hashed pseudonyms.
Input validation: All API inputs validated by Pydantic v2. SQL injection impossible via SQLAlchemy ORM parameterised queries.	**DPDP Act 2023 compliance**: India's Digital Personal Data Protection Act requires data fiduciary registration. Log consent records in consent_log table.
DTC clear gate: Mode 04 (clear codes) requires explicit double-confirmation in UI and audit log entry. Cannot be called by API without OWNER role.	**Vulnerability disclosure**: Add security.txt to ignitionai.in from day 1. Respond to reports within 72 hours.

© 2026 IgnitionAI — Confidential & Proprietary

12. Technology Stack Master Table

Layer	Technology	Version	Rationale
Mobile	Flutter + Dart	3.22+	Single codebase, strong BT packages, APK deploy
Mobile OBD	flutter_bluetooth_serial	latest	RFCOMM socket to ELM327
Mobile DB	Drift (SQLite)	2.x	Type-safe offline ORM for Flutter
Mobile State	Riverpod	2.x	Simpler than BLoC, good DX for 2-person team
Backend	FastAPI	0.111+	Async, auto OpenAPI, Pydantic native
Backend Lang	Python	3.11	Ecosystem for ML, OBD, automotive libs
ORM	SQLAlchemy 2.0 + Alembic	2.0 / 1.13	Migrations + async ORM
Validation	Pydantic v2	2.x	10x faster than v1, native FastAPI integration
Primary DB	PostgreSQL	16	JSONB for flexible PID storage, proven reliability
Cache	Redis 7	7.x	Session state, LLM response cache, rate limiting
Web Frontend	React 18 + TypeScript	18 / 5.x	Dashboard for workshop owners
Web UI Lib	shadcn/ui + Tailwind	latest	Fast, accessible components
Web State	TanStack Query	5.x	Server state management, auto-refresh
LLM	OpenAI GPT-4o-mini / Claude Haiku	latest API	Low-cost inference, JSON mode
CI/CD	GitHub Actions	—	Free for public repos, tight GitHub integration
Infra	Railway.app	—	Zero-config Docker deploy, managed PG included
Monitoring	Sentry + Grafana Cloud	—	Error tracking + metrics, both free tier
Testing	pytest + httpx + factory_boy	latest	Async test client for FastAPI

© 2026 IgnitionAI — Confidential & Proprietary

13. Eight-Week Build Roadmap

Each week has a defined deliverable that can be demonstrated independently. Never begin Week N+1 without the Week N demo working end-to-end. Scope ruthlessly — cut features, not quality.

Week	Theme	Backend Deliverables	Mobile Deliverables	Test Gate
1	Foundation	FastAPI skeleton, auth endpoints, DB migrations, health check deployed on Railway	Flutter project setup, BT permission scaffold, login screen	Can POST /auth/token and get JWT
2	OBD Pipeline	OBD data ingest endpoint, PID decoder module, FaultSession model	BT device scan + ELM327 connect, AT command sequence, raw DTC read	App connects to real dongle, reads 3 DTCs
3	Data Layer	Full PID capture endpoint, vehicle CRUD, DTC knowledge base seed (top 50 codes)	New session flow, symptom intake form, vehicle picker, offline SQLite sync	Complete FaultSession JSON stored in DB
4	Rule Engine v1	DiagRule dataclass + runner, 30 rules across 5 DTC clusters, DiagOutput model	Diagnosis result screen, ranked card UI, confidence bars	P0300 input → correct hypothesis output in < 2s
5	Test Plans	TestStep generator for all 30 rules, parts list builder	Test step checklist screen, pass/fail capture, photo note (camera)	End-to-end: OBD scan → diagnosis → step 1 displayed
6	Bayesian + Feedback	Bayesian updater module, feedback endpoint, outcome label job	Feedback submission screen, session history list	Feedback updates prior; 2nd scan on same car shows improved confidence
7	LLM + Polish	LLM integration (GPT-4o-mini), PDF report generator, web dashboard skeleton	PDF export, web view in-app, Hindi string localisation (static)	Complex case (low rule confidence) returns LLM hypothesis
8	Hardening + Demo	Load testing (Locust, 50 concurrent), error handling pass, API docs review	UI polish, offline mode testing, APK build + TestFlight Android	Full end-to-end demo on 3 different vehicles without crashes

© 2026 IgnitionAI — Confidential & Proprietary

14. Team Responsibilities & Handoff Protocol

With 2 interns at 12–15 hours/week and you as the lead, clear ownership is critical. Interns should never be blocked waiting for you. Design tasks to be independently completable with clear acceptance criteria.

You (Shashank) — Product Engineering Lead	Intern 1 — Backend / Data Pipeline
Diagnostic rule engine (Section 5.1) — only you have the automotive domain knowledge	FastAPI boilerplate, auth module, CRUD endpoints
Bayesian module logic and evidence weight design	OBD decoder integration (python-OBD library)
API endpoint design and schema decisions	Database migrations with Alembic
All acceptance criteria definition for intern tasks	Unit tests for all backend modules
Code review for all PRs before merge to dev branch	Docker setup and Railway deployment config
Weekly demo preparation and stakeholder updates	
LLM prompt engineering and guardrail design	**Intern 2 — Mobile / Frontend**
Database schema decisions (never delegate schema changes)	Flutter app screens (all UI from wireframes you provide)
	BT serial connection and AT command implementation
	Drift offline database schema and sync logic
	PDF report generation module
	Web React dashboard (Week 7–8 only)

14.1 Handoff Protocol

- **Task format**: Every task assigned as a GitHub Issue with: description, acceptance criteria, time estimate, linked design doc section. No verbal task assignments.
- **PR requirement**: All code via Pull Request. Minimum 1 review (yours) before merge. Auto-lint on PR via GitHub Actions.
- **Daily async standup**: Each intern posts a WhatsApp/Slack message: 'Done yesterday / Doing today / Blocked on'. You unblock within 4 hours.
- **Weekly sync**: 45-minute video call every Monday. Demo what was built. Adjust scope for the week.
- **Definition of done**: Feature is done when: code merged, unit test written, manually verified on a real device (not just emulator for mobile), and acceptance criteria checked off.

© 2026 IgnitionAI — Confidential & Proprietary

15. Known Risks & Engineering Mitigations

Risk	Probability	Impact	Mitigation
ELM327 dongle quality varies — cheap dongles drop BT connection or return garbage data	High	High	Test with 3 different dongles during Week 2. Implement retry logic (3 attempts) and connection health check every 30s. Recommend specific verified dongle model in onboarding.
Vehicle does not respond to OBD commands (pre-2005 cars, some 2-wheelers)	Medium	Medium	Detect non-response within 5s, show clear error 'Vehicle OBD not supported', offer manual symptom-only mode. Never hang the UI.
Rule engine returns no hypothesis (DTC not in knowledge base)	High	Medium	Implement graceful fallback: show DTC definition + generic 'top checks' from KB. Never show empty results. Log to unknown_dtcs table for knowledge base expansion.
Intern drops out or is unproductive	Medium	High	Keep backend and mobile as fully independent modules with no circular dependencies. If one intern leaves, the other can continue. You can cover either module at reduced pace.
Railway free tier limits cause API throttling during demo	Low	High	Upgrade to Railway $5/mo Starter plan before any external demo. Keep a Render.com fallback deployment ready.
LLM API returns invalid JSON breaking parser	Medium	Medium	Wrap all LLM calls in try/except. Validate output against Pydantic schema. Fall back to rule engine result if parsing fails. Log all LLM failures to Sentry.
Diagnostic accuracy too low for real workshop use	Medium	High	Be transparent: show confidence scores. Frame MVP as 'diagnostic assistant' not 'diagnostic authority'. Set expectation with pilot workshops: 70% accuracy is the target, rising with feedback data.

Appendix: Recommended Reading & References

© 2026 IgnitionAI — Confidential & Proprietary

- SAE J1979 — OBD-II standard for diagnostic test modes and PIDs
- SAE J2012 — Diagnostic Trouble Code definitions (powertrain, chassis, body)
- ISO 15031 — Road vehicles OBD communication (international equivalent of SAE J1979)
- python-OBD library documentation: https://python-obd.readthedocs.io
- ELM327 AT command datasheet: ELM Electronics v2.2
- FastAPI documentation: https://fastapi.tiangolo.com
- Drift (Flutter SQLite ORM): https://drift.simonbinder.eu
- Bayesian networks for fault diagnosis: 'Bayesian Networks and Decision Graphs' — Jensen & Nielsen
- India automotive service market: ACMA Annual Report 2024
- DPDP Act 2023 compliance guide: MeitY official documentation

End of IgnitionAI Technical Architecture Blueprint v1.0 — Confidential

© 2026 IgnitionAI — Confidential & Proprietary